Guía de Instalación y Configuración

PetCare, Sistema Veterinario

Aplicación Expo/React Native, API Express/TypeScript y base MariaDB/MySQL

Asignatura: Proyecto, Diseño e Implementación de Sistemas Computacionales

Grupo: A 7°4°

Alumna: Banegas Silvia Agustina

Contenido

1. Introducción
2. Requisitos previos
3. Clonación e instalación
4. Configuración del frontend
5. Configuración del backend
6. Base de datos Layerbase
7. Autenticación y cuentas
8. Notificaciones
9. Configuración de Vercel
10. Generación del APK Android
11. Verificaciones posteriores
12. Solución de problemas
13. Migraciones y verificación del esquema (solo lectura)

1. Introducción

Esta guía explica cómo instalar y ejecutar PetCare en un equipo local y cómo preparar las integraciones que se usan en desarrollo: base de datos, inicio de sesión con redes sociales, correo, notificaciones y despliegue.

El sistema tiene dos partes: un cliente Expo/React Native en la carpeta mobile/ y una API Express/TypeScript en la carpeta backend/. La API necesita una base MariaDB/MySQL accesible; los despliegues descritos en el repositorio usan Layerbase, y en desarrollo local se puede usar una base propia.

Las migraciones no se ejecutan automáticamente al iniciar la aplicación. Deben aplicarse de forma manual, después de revisar el SQL (ver la sección 3).

Arquitectura

Componente	Ubicación	Responsabilidad y tecnologías
Aplicación cliente	`mobile/`	Expo SDK 58, React Native 0.88, React 19, TypeScript 6 y Expo Router 58. Exporta web y compila Android.
API	`backend/`	Node.js 22 recomendado, Express 5, TypeScript 5.9 y JWT. Organizada en rutas, controladores, servicios, repositorios, middleware y validadores.
Datos	Layerbase	Servicio remoto compatible con MariaDB/MySQL. El backend usa `mysql2/promise`, pool SQL y TLS en producción.
Despliegue	Vercel / EAS	Vercel sirve dos proyectos separados; EAS Build compila Android.

El frontend obtiene el rol y los datos de la API, pero la autorización final se valida en el backend. El esquema de carpetas relevante es `mobile/src/{app,components,contexts,hooks,services,utils,types}` y `backend/src/{routes,controllers,middlewares,services,repositories,validators,config,database}`.

2. Requisitos previos

Las versiones siguientes proceden de los manifiestos actuales. No se fija `engines.node` en los `package.json`; Node 22.x y npm son los recomendados por los README, no una restricción impuesta por npm.

Herramienta	Requisito observado
Node.js	22.x recomendado. Usar una versión compatible con Expo SDK 58 y TypeScript de cada paquete.
npm	Necesario; se incluyen `package-lock.json`. Preferir `npm ci` para instalaciones reproducibles. No se fija versión de npm.

Herramienta	Requisito observado
Git	Necesario para clonar y obtener el historial/fuente. No se fija versión.
Expo / React Native	Frontend declara `expo ~58.0.16`, `expo-router ~58.0.24`, `react-native 0.88.0`, `react 19.3.0`. No hace falta instalar Expo CLI global: `npm start` usa el paquete local.
EAS CLI	`eas.json` requiere CLI `>=16.0.0`; los comandos de esta guía usan `npx eas-cli@latest`. Cuenta de Expo/EAS necesaria para builds cloud.
Android	Android Studio y Android SDK para `expo run:android` local. Un teléfono Android para instalar el APK. Para EAS remoto no hace falta compilar localmente, pero sí cuenta EAS y credenciales de firma.
Navegador	Navegador moderno para la salida web de Expo.
Base SQL	MySQL 8.0.16 o superior, MySQL 9.x o MariaDB 11.8.x según el chequeo del backend. Layerbase es el servicio de producción esperado.
Correo	Servidor SMTP si se habilitan registro, verificación, invitaciones o recuperación de contraseña.
OAuth	Credenciales de Google, Facebook y/o X únicamente si se habilitan esos proveedores.
Imágenes	Cloudinary o almacenamiento compatible S3 si se habilita carga de fotos.
Cron	Un programador externo/configuración válida de Vercel para invocar el endpoint, si se desean recordatorios y cola de correo.

Revisá versiones y compatibilidad en `backend/package.json`, `mobile/package.json`, `mobile/app.json` y `mobile/eas.json`; no sustituyas versiones a partir de esta guía sin comprobar compatibilidad. Las capacidades opcionales (login social, SMTP, fotos, cron) solo se necesitan si se habilita esa función.

3. Clonación e instalación

```
git clone <URL_DEL_REPOSITORIO>
cd PetCare-SistemaVeterinario
```

Instalá y arrancá el backend desde `backend/`:

```
cd backend
npm ci
cp .env.example .env
# Editar .env localmente con la configuración del entorno autorizado.
npm run dev
```

En otra terminal, desde la raíz del repositorio instalá el cliente:

```
cd mobile
npm ci
cp .env.example .env
npm start
```
En Windows PowerShell, para copiar plantillas se puede usar `Copy-Item .env.example .env`. Los archivos `.env` locales no deben subirse ni copiar secretos reales a la documentación. Desde `backend/`, `npm run build` compila a `dist/` y `npm start` sirve `dist/server.js`. Desde `mobile/`, los comandos disponibles son `npm start`, `npm run web`, `npm run android`, `npm run typecheck`, `npm run lint` y `npm test`.

La API local escucha por defecto en `http://localhost:3000/api`. La app en un teléfono físico debe usar la IP LAN del equipo; para emulador Android puede usarse `http://10.0.2.2:3000/api`.

4. Configuración del frontend

Estructura y rutas

`mobile/src/app/` usa Expo Router y layouts para sesión y áreas por rol. Servicios de red y autenticación están en `mobile/src/services/`; contexto de sesión/tema/checkout en `mobile/src/contexts/`. Los guards del cliente controlan navegación y experiencia, pero el backend vuelve a validar permisos.

URL de API y variables Expo

Variable	Uso
`EXPO_PUBLIC_API_URL`	URL base configurada para web/EAS. Debe apuntar al backend e incluir `/api`. En los perfiles EAS de producción está fijada a `https://pet-care-sistema-veterinario.vercel.app/api`.
`EXPO_PUBLIC_DEV_API_URL`	Sobrescritura explícita HTTPS para desarrollo; tiene prioridad cuando `__DEV__` está activo. Debe terminar en `/api`.
`EXPO_PUBLIC_API_PORT`	Puerto local cuando el cliente deriva el host LAN desde Metro; por defecto `3000`.
`EXPO_PUBLIC_APP_ENV`	Entorno `development`, `preview` o `production`, usado por el resolver y los perfiles EAS.

El resolver de `mobile/src/services/api.ts` usa primero `EXPO_PUBLIC_DEV_API_URL` en desarrollo. Si no está, utiliza `EXPO_PUBLIC_API_URL` cuando corresponde o deriva el host LAN de Metro y el puerto indicado. En builds de producción no usar la variable `DEV`. Todas las variables `EXPO_PUBLIC_*` quedan incluidas en el cliente; nunca guardar secretos ahí.

Ejemplos de URL local:

```
EXPO_PUBLIC_APP_ENV=development
EXPO_PUBLIC_API_URL=http://10.0.2.2:3000/api
EXPO_PUBLIC_API_PORT=3000
```
Para conectar un dispositivo a una API de desarrollo remota:

```
EXPO_PUBLIC_DEV_API_URL=https://<dominio-del-backend>/api
```

No omitas `/api`: las rutas registradas son, por ejemplo, `/api/auth/login` y `/api/client/pets/query`. Sin el prefijo, Vercel/web puede contestar 404 aunque el host sea correcto. Tampoco uses el dominio del frontend como host de API.

Para un teléfono físico usá la IP LAN de la computadora, en la misma red y con el firewall habilitado para el puerto; para el emulador Android, `10.0.2.2`. El backend debe aceptar el origen web local en `CORS_ALLOWED_ORIGINS` y el dispositivo debe poder alcanzar su host.

Web, Android y Expo Go

- Web: desde `mobile/`, `npm run web`. `mobile/vercel.json` exporta `expo export -p web` a `dist` y configura rewrite de rutas.
- Android local: desde `mobile/`, `npm run android`; requiere Android Studio/SDK, dispositivo o emulador configurado.
- Expo Go: válido para explorar pantallas y login tradicional si el cliente Expo Go instalado es compatible con el SDK del repo. El retorno OAuth `petcare://oauth` requiere una development build; Expo Go explica esa limitación.
- Development build: `npx expo start --dev-client`. Cambiar plugins, scheme o configuración nativa requiere reconstruir el cliente nativo.
- El scheme Android configurado es `petcare`, el paquete `com.anonymous.petcare`; el `projectId` EAS se declara en `mobile/app.json`.

El OAuth social usa el navegador del sistema y callbacks de servidor. Google está integrado con flujo servidor code + PKCE; el proyecto no implementa el SDK Android de Google Sign-In. Las consolas de cada proveedor deben tener el callback HTTPS del backend permitido. No se pueden verificar las credenciales reales de proveedor a partir del repositorio.

5. Configuración del backend

Variables

La plantilla `backend/.env.example` documenta nombres y defaults. Configurá según la función que vayas a operar:

Grupo	Variables	Uso / validación
Proceso	`NODE_ENV`, `PORT`	`development`, `test` o `production`; puerto predeterminado 3000.
SQL	`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL_MODE`, `DB_CONNECTION_LIMIT`, `DB_CONNECT_TIMEOUT_MS`	Producción requiere host remoto, usuario, contraseña y TLS (`required` o `verify_identity`). Los límites y puertos se validan. El nombre de base admite letras, números y `_`.
JWT/sesión	`JWT_ACCESS_SECRET`, `JWT_ACCESS_TTL_SECONDS`, `JWT_REFRESH_TTL_SECONDS`, `JWT_ISSUER`, `JWT_AUDIENCE`, `BCRYPT_SALT_ROUNDS`	Secreto access de al menos 32 bytes; TTL access predeterminado 900 s y máximo 3600; refresh predeterminado 30 días; bcrypt entre 10 y 14 rondas (default 12).

Grupo	Variables	Uso / validación
Correo	`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_REQUIRE_TLS`, `SMTP_USER`, `SMTP_PASSWORD`, `MAIL_FROM`, `EMAIL_CODE_SECRET`	SMTP requiere remitente válido y TLS seguro; usuario/clave deben configurarse juntos. El secreto de códigos requiere 32 bytes.
URLs / OAuth	`PUBLIC_FRONTEND_URL`, `BACKEND_URL`, `OAUTH_RETURN_URLS`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `FACEBOOK_CLIENT_ID`, `FACEBOOK_CLIENT_SECRET`, `FACEBOOK_GRAPH_VERSION`, `X_CLIENT_ID`, `X_CLIENT_SECRET`	Frontend público para enlaces; origen backend para callbacks. Facebook requiere versión Graph `vN.N`. X admite también variables heredadas `TWITTER_CLIENT_ID` y `TWITTER_CLIENT_SECRET`.
Acceso / cron	`CORS_ALLOWED_ORIGINS`, `CRON_SECRET`	Lista CSV de orígenes exactos HTTPS en producción. `CRON_SECRET` debe tener al menos 32 bytes.
Fotos	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`; o `ACCESS_KEY_ID`, `SECRET_ACCESS_KEY`, `REGION`, `BUCKET`, `ENDPOINT`, `PET_PHOTO_S3_URL_STYLE`	Configuración privada de Cloudinary o S3 compatible. No poner estas claves en el cliente.
Opcionales	`STORE_DEMO_MODE`, `REGISTER_DIAGNOSTICS`	Checkout de demo desactivado por defecto; diagnósticos de registro solo en entorno no productivo.

Los nombres `PETCARE_EXPECTED_DB_HOST` y `PETCARE_EXPECTED_DB_NAME` también son leídos por el preflight de migraciones de solo lectura. No aparecen en `.env.example` porque son protecciones adicionales de ese comando. `VERCEL`, `VERCEL_ENV`, `VERCEL_REGION`, `AWS_LAMBDA_FUNCTION_NAME` y `LAMBDA_TASK_ROOT` son indicadores del runtime/plataforma que el código detecta; normalmente los define el host y no son secretos de aplicación.

Arranque, CORS y errores

El arranque es `npm run dev`; no aplica migraciones al iniciar. `npm run build` compila y `npm start` inicia el servidor compilado. `/api/health` informa estado del proceso y `/api/health/database` prueba consulta SQL. En producción CORS valida orígenes HTTPS exactos; además se admite el dominio oficial y previews bajo el patrón configurado. El cliente Android nativo no depende de CORS de navegador.

JWT identifica la sesión; el middleware vuelve a cargar usuario, rol, versión de sesión y permisos efectivos. Las rutas pueden exigir roles, todos o alguno de los permisos listados y ownership. No es suficiente ocultar botones en el frontend.

Comandos del backend

Comando	Resultado
`npm run dev`	Servidor de desarrollo con recarga.
`npm run build`	Compila TypeScript a `dist/`.
`npm start`	Ejecuta `dist/server.js`.
`npm run typecheck`, `npm run lint`, `npm test`	Tipos, ESLint y pruebas unitarias.
`npm run oauth:callbacks`	Muestra los callbacks de proveedores según `BACKEND_URL`.
`npm run db:check`, `npm run db:audit:readonly`	Verificaciones de base/SQL de solo lectura.

El paquete define además comandos de migración, bootstrap, semillas, creación/promoción de administradores y pruebas con base de datos. Revisá `backend/package.json` antes de invocarlos: algunos modifican datos o esquema. Los logs del backend están sanitizados y no muestran SQL completo, parámetros ni tokens.

6. Base de datos Layerbase

El backend conecta a MySQL/MariaDB mediante un pool `mysql2`, codificación UTF-8 y timezone SQL UTC. En Vercel el pool reduce su límite predeterminado a 1 por instancia para no multiplicar conexiones. Producción rechaza loopback/IP privadas y exige TLS. No se incluye una instancia de base local ni datos de credenciales.

Conexión a Layerbase / MariaDB

Obtené host, puerto, nombre de base, usuario, contraseña y requisitos TLS desde el panel de Layerbase; no supongas que `localhost` apunta a una base remota. Usá `DB_SSL_MODE=required` (o `verify_identity` cuando el host y el certificado lo permiten). Producción rechaza loopback, IP privada de LAN y hosts Railway heredados. Con `backend/.env` completo, `npm run db:check` y `npm run db:audit:readonly` son comprobaciones de solo lectura; en un entorno desplegado, `GET /api/health/database` prueba la conexión.

Migraciones y precauciones

Los comandos de migración y bootstrap definidos en package.json pueden modificar el esquema o los datos. El checkout no incluye el SQL canónico necesario para instalar una base limpia y la gestión del esquema productivo es externa. No ejecutes esos comandos como instalación; usa únicamente el procedimiento autorizado por el responsable de la base.

Para diagnóstico no mutante, ejecutar desde `backend/`:

```
npm run db:check
npm run db:audit:readonly
```

`db:check` verifica conexión y versión compatible; la auditoría de solo lectura examina esquema. Para verificar migraciones remotas, consultar al responsable cuál es el repositorio/proceso operativo y comparar historial/esquema de Layerbase. No copiar SQL antiguo ni modificar/borrar tablas con datos compartidos sin respaldo, revisión de impacto y plan aprobado.

Instalación desde cero: el checkout no incluye el SQL base necesario para crear el esquema. Aunque package.json declara comandos de bootstrap y migración, npm ci no instala la base de datos y esos comandos no sustituyen el paquete canónico. Antes de crear una base nueva, obtené del responsable el procedimiento soportado. La app no corrige el esquema automáticamente al iniciar.

7. Autenticación y cuentas

- Registro público crea una cuenta `CLIENT` pendiente de verificar; no acepta rol del usuario en el formulario.
- El login devuelve access y refresh token solo a cuentas activas, verificadas y que no requieran cambio inicial de contraseña.
- El access JWT vence en 900 s por defecto; refresh se almacena como sesión revocable y dura 30 días por defecto. `POST /api/auth/refresh` rota/renueva la sesión y `/logout` revoca el refresh enviado.
- La verificación y recuperación envían correo mediante SMTP. La contraseña se procesa con bcrypt; invitaciones/restablecimientos usan tokens aleatorios de un uso con hash almacenado y vencimiento.
- Roles observados: `CLIENT`, `VETERINARIAN`, `GROOMER`, `ADMIN`, `SUPER_ADMIN`. Los permisos vienen de la base y el middleware los aplica en cada request. No hay rol actual `SECRETARY`.
- Invitaciones: el administrador crea la cuenta/invitación según permisos; el backend construye el enlace con `PUBLIC_FRONTEND_URL`, envía el token por correo y la invitación vence en 24 h. `POST /api/auth/invitations/accept` establece la contraseña inicial, activa/verifica la cuenta e invalida sesiones anteriores. `PUBLIC_FRONTEND_URL` es el sitio público que recibe el enlace; `BACKEND_URL` es el origen público de API usado para callback OAuth. No son intercambiables.
- Google, Facebook y X son proveedores configurables; availability indica si están configurados para el `returnUrl` solicitado. Las credenciales se mantienen exclusivamente en backend.
- La baja de usuario de administración es lógica y revoca sesiones; no es borrado físico. La eliminación de rol es solo para roles custom sin usuarios asignados.

Configurar OAuth externo (Google, Facebook, X)

El login JWT tradicional usa `POST /api/auth/login`; el frontend conserva la sesión en almacenamiento seguro nativo y en `sessionStorage` en web, y no guarda la contraseña. Para el login social:

1. Configurá la app OAuth web en la consola del proveedor y habilitá el flujo que pide su producto.

2. En el backend cargá los client ID/secret privados y `BACKEND_URL` (origen sin `/api`).
3. Definí `OAUTH_RETURN_URLS` con los destinos finales permitidos, como la página web `/oauth` y el esquema nativo `petcare://oauth` según el entorno.
4. Ejecutá `npm run oauth:callbacks` desde `backend/` y copiá al portal del proveedor el callback HTTPS exacto: `/api/auth/oauth/{provider}/callback`.
5. Verificá dominio, URI de retorno, modo de publicación y permisos en la cuenta de desarrollador del proveedor.

Google y X usan PKCE; Facebook usa el flujo configurado en el backend. PetCare no vincula identidades automáticamente porque compartan email y exige verificar el correo en las altas que lo requieran. X no entrega email en el flujo descrito por el código. No pongas secretos OAuth en Expo, `mobile/.env`, `EXPO_PUBLIC_*` ni en el repositorio.

8. Notificaciones

El centro de notificaciones es interno y persistido en base de datos; admite consulta, lectura, lectura total y ocultamiento. Existe alias bajo `/api/client/notifications` y ruta compartida `/api/notifications`, sujeta al guard de acceso de notificaciones. No hay Expo Push ni Web Push.

Correo (SMTP)

La verificación, invitación y recuperación por correo dependen de un relay SMTP alcanzable, credenciales activas, un remitente autorizado y URLs de frontend válidas. `SMTP_SECURE` y `SMTP_REQUIRE_TLS` deben coincidir con lo que exige el proveedor. Si no llega un correo, revisá los logs del proveedor y la carpeta de spam; el endpoint público no debe usarse para inferir si una cuenta existe.

El servicio `processNotificationJobs` crea recordatorios internos para turnos confirmados con ventanas de 24 h y 4 h y procesa la cola de correo. Los correos fallidos reintentan con espera exponencial acotada y hasta 8 intentos; se conservan estados como `PENDING`, `SENDING`, `SENT`, `FAILED`. El endpoint `GET /api/cron/notifications` exige `Authorization: Bearer <CRON_SECRET>`.

El archivo `backend/vercel.json` está vacío en el checkout: no declara `crons`. Aunque hay endpoint y lógica listos, el repositorio no demuestra que Vercel lo invoque automáticamente. Configurá una fuente de cron compatible y su autorización en el entorno real; verificá frecuencia, plan y logs en Vercel antes de contar con recordatorios/correos automáticos.

9. Configuración de Vercel

Hay dos proyectos esperados (confirmar en Vercel, ya que el repositorio no consulta el estado remoto):

Proyecto	Dominio esperado	Root Directory	Build observado
Frontend	`https://petcare-sistema veterinario-mobile.verc el.app`	`mobile`	`npx expo export -p web`; output `dist`, según `mobile/vercel.json`.
Backend	`https://pet-care-sistem a-veterinario.vercel.ap p`	`backend`	API Express/TypeScript detectada por Vercel. `backend/vercel.json` actualmente es `{}`.

En Vercel, revisar que cada proyecto apunte al mismo repositorio, Root Directory correcto y deployment del commit esperado. Mantener builds automáticos habilitados o lanzar el deploy de un commit nuevo. Para frontend los valores públicos de producción se encuentran en `mobile/vercel.json`; para backend cargar como variables **privadas** los parámetros SQL, JWT, frontend, SMTP, CORS, OAuth, cron y almacenamiento que correspondan. Configurar `PUBLIC_FRONTEND_URL` con el origen web esperado y `BACKEND_URL` con el origen backend sin `/api` (el callback agrega `/api/auth/oauth/<proveedor>/callback`). `CORS_ALLOWED_ORIGINS` debe incluir el origen web exacto.

Antes de dar por publicado:

1. En Deployments, confirmar rama/commit y que el deployment figure **Ready**.
2. Verificar el dominio asignado al proyecto, no inferirlo desde el nombre del repositorio.
3. Probar `https://pet-care-sistema-veterinario.vercel.app/api/health` y `/api/health/database`.
4. En navegador, recargar la app web y confirmar que el bundle solicita la API con `/api`.
5. Inspeccionar Build Logs y Runtime/Function Logs para errores 404, DB, SMTP u OAuth.

Redeploy vuelve a construir el commit seleccionado con sus variables actuales; no incorpora cambios de Git que no estén en ese commit. Para publicar código nuevo, crear/push del commit y confirmar un nuevo deployment (o elegir explícitamente el commit nuevo). Para descartar un bundle móvil viejo, comprobar URL y commit, limpiar la caché de build si procede y generar un nuevo build/export; una instalación Android ya descargada no recibe cambios JS solo porque Vercel haya cambiado.

10. Generación del APK Android

El perfil pedido existe en `mobile/eas.json`: `internal-production-api`, `distribution: internal`, entorno EAS `production`, `buildType: apk`, incremento automático y URL API productiva terminada en `/api`.

Otros perfiles de `mobile/eas.json` mencionados en la guía anterior (confirmalos en el archivo; el verificado en el checkout actual es `internal-production-api`):

Perfil	Resultado
`development`	Development client (APK).
`preview`	APK interno.
`production`	Distribución de producción habitual.
`production-apk`	APK instalable.

Desde la carpeta `mobile/`:

```
npx eas-cli@latest login
npx eas-cli build -p android --profile internal-production-api
```

Requisitos: cuenta EAS autenticada, proyecto EAS enlazado (el `projectId` está en `app.json`), credenciales de firma Android configuradas en EAS y variables del entorno `production`. EAS puede administrar/crear las credenciales de firma al solicitarlo; conservarlas de forma segura para futuras actualizaciones. El perfil genera APK interno instalable; descargar el artefacto del build completado y habilitar instalación desde esa fuente en el dispositivo.

`google-services.json` es el archivo de configuración cliente de Firebase/Google Services para integración Android; una clave JSON de cuenta de servicio es una credencial privada de servidor con permisos IAM. No son intercambiables. El manifiesto actual no declara `google-services.json` ni un plugin Firebase. No cargar claves de cuenta de servicio en el bundle móvil.

Errores habituales: versión EAS CLI demasiado vieja; proyecto EAS no vinculado; variables faltantes; credenciales de firma no configuradas; configuración nativa inválida; módulos nativos nuevos sin reconstruir; cuota/concurrencia de EAS. Consultar logs del build para la primera falla concreta. Un APK exitoso no prueba conectividad SQL, SMTP ni OAuth.

Una development build se sirve con `npx expo start --dev-client`. Los perfiles `preview` y `production-apk` usan la API HTTPS declarada en el perfil, así que comprobá que sea la deseada; para Google Play usá el perfil con el formato de distribución de producción. Instalar EAS CLI con `npx` no agrega dependencias al proyecto. Expo Go sirve para explorar pantallas y el login tradicional, pero no reproduce el esquema nativo OAuth `petcare://oauth`: para esa capacidad usá una development build o un APK propio.

11. Verificaciones posteriores

- ☐ API arranca y `/api/health` responde.
- ☐ `/api/health/database` puede conectar; las auditorías son de solo lectura.
- ☐ Frontend apunta al host correcto de API y conserva `/api`.
- ☐ Login con cuenta activa y verificada; refresh y logout revocan sesión como se espera.
- ☐ Registro de cliente y verificación por correo funcionan con SMTP autorizado.
- ☐ Invitación nueva y aceptación dentro del plazo; enlaces usan `PUBLIC_FRONTEND_URL` vigente.
- ☐ Roles y permisos se comprueban desde el backend.
- ☐ Mascotas: listado, alta y edición para cuenta propietaria.
- ☐ Turnos: catálogo, disponibilidad, alta y cancelación según permisos.
- ☐ Fichas clínicas y vacunas visibles solo con alcance/permisos correspondientes.
- ☐ Tienda, carrito y pedidos; comprobar por separado transferencia/demo. La demo no cobra.
- ☐ Notificaciones internas y estados; confirmar cron por separado antes de esperar recordatorios/correos.
- ☐ APK de perfil indicado instala, abre y contacta la URL de API configurada.
- ☐ Una consulta de lectura `POST` con JSON y una escritura permitida funcionan; se confirma ownership.
- ☐ Desde `backend/` y `mobile/`, `npm run typecheck`, `npm run lint` y `npm test` pasan.
- ☐ Si están configurados, OAuth, fotos y correo se prueban con cuentas controladas.

12. Solución de problemas

Error / síntoma	Causa probable	Diagnóstico	Solución
404 en toda la API	Se omitió `/api` o se llamó una ruta retirada	Probar `/api/health`; revisar la URL de `EXPO_PUBLIC_API_URL`	Configurar base como `<origen-backend>/api`; usar rutas del inventario API vigente.

Error / síntoma	Causa probable	Diagnóstico	Solución
Invitación llega con dominio viejo	`PUBLIC_FRONTEND_URL` tiene valor anterior o ambiente incorrecto en Vercel	Revisar variable del proyecto backend y enlace del correo sin exponer token	Actualizar origen web público y emitir invitación nueva.
Frontend llega a host incorrecto	Env de Vercel/EAS equivocado o `DEV_API_URL` local	Inspeccionar configuración del build/deployment	Confirmar `EXPO_PUBLIC_API_URL` incluye `/api`; recompilar/exportar.
Se ejecuta bundle viejo	Deployment apunta a commit anterior, caché o APK instalado antiguo	Ver commit y estado Ready; revisar URL en logs de red	Publicar commit nuevo; reconstruir web/APK y reinstalar el artefacto actualizado.
`503 DB_UNAVAILABLE`	Red, host, usuario, clave, base o TLS	Revisar runtime logs sanitizados y `/api/health/database`	Corregir variables Layerbase autorizadas, firewall y TLS; no mostrar secreto en logs.
Layerbase no conecta	DNS/puerto restringido, credenciales inválidas o host legado	`npm run db:check`; revisar DB_HOST/DB_PORT y estado del proveedor	Usar host vigente y TLS `required`/`verify_identity`; confirmar allowlist de red.
`401`	Token ausente/vencido/revocado o credenciales incorrectas	Revisar `Authorization: Bearer` y estado de sesión	Iniciar login o refresh válido; credenciales de login usan mensaje genérico.
`403`	Email sin verificar, cambio inicial pendiente, rol/grant o alcance insuficiente	Revisar respuesta y permisos de la cuenta en fuente administrativa	Verificar correo/cambiar contraseña; pedir grant/rol correcto. No confiar en el frontend.
Invitación ya no acepta token	Venció (24 h), fue usada/rotada o se copió incompleta	Revisar estado en flujo de invitaciones sin exponer token	Un administrador vuelve a emitirla; la nueva URL se genera desde frontend público configurado.
Caché de Metro / cambios sin efecto	Bundle o transform cache local	Reiniciar Metro y observar URL del bundle/API	`npx expo start --clear` desde `mobile/`; volver a abrir app.

Error / síntoma	Causa probable	Diagnóstico	Solución
EAS Build falla	CLI, credenciales, variable/env o config nativa	Leer el primer error en Build Logs de EAS	Usar perfil/entorno correcto, autenticar EAS y configurar firma; reconstruir tras cambios nativos.
No llega correo	SMTP/MAIL_FROM/EMAIL_CODE_SECRET ausente, TLS o proveedor bloquea envío	Runtime logs, variables del backend y consola SMTP	Completar SMTP seguro, usuario y clave juntos, remitente válido; intentar nuevo código/invitación.
Recordatorios no aparecen	No hay invocación cron, falta `CRON_SECRET` o esquema/DB no disponible	Revisar cron configurado en plataforma, respuesta del endpoint y logs	Configurar un scheduler real; `backend/vercel.json` del repo no contiene schedule. Enviar Bearer secreto de forma segura.
CORS bloquea web	Origen exacto no permitido	Comparar `Origin` del navegador con variables y patrón permitido	Añadir origen HTTPS exacto en `CORS_ALLOWED_ORIGINS`; evitar paths y credenciales en producción.
`SCHEMA_UPDATE_REQUIRED`	Consulta refiere columna/tabla ausente	Runtime logs muestran código/ruta sin SQL sensible	Comparar esquema con el proceso de migración autorizado; no correr scripts a ciegas.
`PHOTO_STORAGE_UNAVAILABLE`	Cloudinary/S3 no configurado	Revisar configuración privada de almacenamiento	Completar el proveedor elegido en backend; nunca poner secretos en Expo.

Otros problemas frecuentes

Síntoma	Causa y solución
API no responde desde el móvil	`localhost` del teléfono no es la computadora: usá la IP LAN, `10.0.2.2` en emulador, puerto abierto y misma red.
`503` en el login	`JWT_ACCESS_SECRET` ausente o corto, o configuración de token inválida: usá un secreto aleatorio de al menos 32 bytes.

Síntoma	Causa y solución
404 al hacer GET de una consulta: las lecturas migradas usan POST. Consulta el inventario de rutas en Documentacion_API_PetCare_revisada.pdf, en docs/.	
OAuth vuelve con error	Verificá el callback del proveedor carácter por carácter, `BACKEND_URL`, el retorno permitido y las variables privadas de ese proveedor.
Expo Go no abre el login social	Limitación del cliente genérico: generá una development build con EAS.
Foto o comprobante rechazado	Formato MIME no soportado, archivo/base64 corrupto o tamaño excedido: límite de 8 MB para fotos y 5 MB para comprobantes.
Ruta o permiso denegado	Confirmá método, prefijo `/api`, rol y grants vigentes en MySQL; ocultar una pantalla no reemplaza la autorización del backend.

13. Migraciones y verificación del esquema (solo lectura)

El código requiere las tablas admin_sensitive_action_audit y pet_lifecycle_history para registrar operaciones administrativas y cambios de estado de mascotas. El repositorio no incluye el SQL canónico de esas tablas ni confirma el esquema remoto. Antes de habilitar esas operaciones, verificá la estructura de Layerbase con consultas de solo lectura a information_schema. Esta guía no afirma que las tablas estén instaladas.

Antes de habilitar operaciones que escriben en esas tablas, revisá únicamente metadatos del entorno y comparalos con las consultas actuales del backend. No ejecutes cambios de datos ni de esquema para comprobarlo.

```
SELECT TABLE_NAME, COLUMN_NAME, COLUMN_TYPE, IS_NULLABLE, COLUMN_DEFAULT,
ORDINAL_POSITION
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE()
  AND TABLE_NAME IN ('pet_lifecycle_history','admin_sensitive_action_audit')
ORDER BY TABLE_NAME, ORDINAL_POSITION;

SELECT TABLE_NAME, INDEX_NAME, NON_UNIQUE, SEQ_IN_INDEX, COLUMN_NAME
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = DATABASE()
  AND TABLE_NAME IN ('pet_lifecycle_history','admin_sensitive_action_audit')
ORDER BY TABLE_NAME, INDEX_NAME, SEQ_IN_INDEX;

SELECT TABLE_NAME, CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME,
REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
  AND TABLE_NAME IN ('pet_lifecycle_history','admin_sensitive_action_audit')
ORDER BY TABLE_NAME, CONSTRAINT_NAME, ORDINAL_POSITION;
```

El backend espera ambas tablas para registrar las operaciones indicadas. La estructura exacta, los índices y las restricciones remotas no se verifican desde este repositorio; confirmalos mediante consultas de solo lectura antes de habilitar estas funciones.

Estado de verificación: el esquema remoto de Layerbase no se confirmó. Ejecutá las consultas de metadatos con una cuenta de solo lectura y anotá cualquier diferencia antes de declarar compatible el entorno. No copies resultados con datos personales: estas consultas solo leen metadatos.

Instalación limpia: este checkout no incluye los scripts SQL base necesarios para crear el esquema. npm ci no instala la base de datos. Para aprovisionar una base nueva, obtené el paquete canónico mediante el procedimiento aprobado por el responsable antes de iniciar la aplicación.

Las URL, secretos, estado de Layerbase, configuración de Vercel y credenciales de proveedores deben verificarse en sus consolas; esta guía no incluye secretos.